

Vendor Estimate Sharing

Last updated: 2026-05-29

Vendors maintain their own estimation matrix (the same stack studios use), optionally vary their rates per studio relationship, and share a **frozen, granularity-controlled snapshot** of those estimates with a linked studio. It is the reverse direction of asset payload dispatch: studio→vendor payloads send asset data; vendor→studio estimate shares send a rate card.

v1 is **visible-only** on the studio side, the studio can read the snapshot, but it is not yet wired into scenario planning. Full design and decision history live in [docs/plans/vendor-estimate-share.md](#).

Concepts

Org-scoped estimation stack. `workflow_steps`, `estimate_config`, and `estimate_matrix` were originally studio-only. They are now owned by **either** a studio or a vendor via a dual nullable foreign key (`studio_id` / `vendor_id` , exactly one set) plus a generated `owner_key` (`coalesce(studio_id, vendor_id)`) that is the single uniqueness arbiter. Studio behaviour is byte-for-byte unchanged, studio rows keep `studio_id` ; vendor rows are new. See [Estimation Engine](#) for the matrix mechanics.

Authoring prerequisite. A vendor builds its base matrix through the same OrgHub **Setup** wizard a studio uses. The wizard's variable/value discovery (`routes/fields.py` , `/fields` , `/field-values` , `/asset-combinations`) is owner-aware: it reads the **org's own** `source_field_mappings` / `replicated_assets` . So a vendor's estimation axes come from the vendor's *own* synced assets, not the studio's, a vendor must have a connected, synced source for the wizard to offer any variables. (There is deliberately no manual-variable entry path; the eventual studio-side reconciliation of vendor vs. studio vocabulary is the deferred Phase 4 work.)

Base matrix + per-link overrides. A vendor keeps one **base** matrix (`link_id IS NULL`) and may override individual cells for a specific studio link (`link_id` set). The **effective matrix** for a link is the base overlaid with that link's overrides, overrides win per (`workflow_step_id` , `variable_values`) . Workflow shape and variable axes stay vendor-global; only the day-count rates diverge per studio.

Granularity (the IP-exposure boundary). At share time the vendor picks how much process detail to reveal. This is the only thing standing between a vendor's internal workflow and the studio:

Granularity	What the studio sees
<code>asset_total</code>	One total per asset profile. No step or craft breakdown.
<code>craft_bucket</code>	Totals grouped by craft. Step estimates summed within each craft; no step names.
<code>workflow_step</code>	Per-step detail, labelled with each workflow step name.

The projector (`lib/estimate/projector.py`) emits strictly what the granularity permits, it is a pure function so the exposure rule is unit-tested without a DB (`scripts/test_projector.py`).

Snapshot. A share is a frozen, self-describing JSON projection of the effective matrix , analogous to a payload dispatch, minus the one-time token and the canonical-asset anchor. Changes to the vendor's matrix after sharing do **not** affect what the studio already received.

Series + replace semantics. A **series** is the sharing channel for one relationship, keyed on (`vendor_id`, `link_id`), exactly one channel per studio link. Granularity is a per-share property of the *dispatch*, not part of the channel key. Re-sharing **replaces** the prior live share regardless of granularity, so dialing disclosure *down* (e.g. `workflow_step` → `asset_total`) genuinely reduces what the studio can read rather than leaving the detailed share live. At-most-one-live is structural (a partial unique index), not a route-layer convention.

Delivery = route-scoped inbox. The studio reads active dispatches addressed to it; the route-layer owner filter (`recipient_studio_id`) is the enforced isolation boundary. Like the rest of the stack, the routes use the service-role client, so RLS is **defense-in-depth** (it only fires under a user JWT, which the app never is), not the runtime guard. Delivery is a pluggable strategy (`delivery_mode` , currently only `route_inbox`); an external token-based strategy is reserved but not built.

Data Model

Org-scoped estimate tables (migration `20260529000001`)

`workflow_steps` , `estimate_config` , and `estimate_matrix` each gain:

```
vendor_id    uuid → vendors          (nullable; studio_id also now nullable)
              check (num_nonnulls(studio_id, vendor_id) = 1)
owner_key    uuid generated always as (coalesce(studio_id, vendor_id)) stored
```

Uniqueness is rebuilt over `owner_key` (single full unique index, also the PostgREST `on_conflict` arbiter): `estimate_config` (`owner_key`), `workflow_steps` (`owner_key`, `airtable_template_id`). The studio-only RLS policies are extended with matching `*_vendor` policies (defense-in-depth). `estimate_config`'s primary key is swapped from (`studio_id`) to a surrogate `id uuid` (verified no inbound FKs).

`estimate_matrix` link overrides (migration `20260529000002`)

```
link_id      uuid → studio_vendor_links  (nullable; NULL = base row, set = override row)
              check (link_id is null or vendor_id is not null)  -- overrides are vendor-only
```

Unique index recreated as (`owner_key`, `workflow_step_id`, `variable_values`, `link_id`) **NULLS NOT DISTINCT** (so two base rows collide). PostgREST `on_conflict` binds this index (validated on Postgres 17).

estimate_share_series (migration 20260529000003)

```
id                uuid PK
vendor_id         uuid → vendors
link_id           uuid → studio_vendor_links
recipient_studio_id uuid → studios      (denormalized for the inbox filter)
label             text
created_at        timestampz
unique (vendor_id, link_id)          -- one channel per relationship
```

estimate_share_dispatches

```
id                uuid PK
series_id         uuid → estimate_share_series
vendor_id         uuid → vendors      (sender, denormalized)
recipient_studio_id uuid → studios    (denormalized for the inbox filter)
link_id           uuid → studio_vendor_links
snapshot          jsonb              -- frozen self-describing projection (carries
delivery_mode     text default 'route_inbox'
expires_at        timestampz         -- nullable: open-ended share allowed
revoked_at        timestampz
superseded_at     timestampz         -- set on re-share (replace semantics)
created_at        timestampz
```

uq_esd_one_live, partial unique index on (series_id) where superseded_at is null and revoked_at is null, makes at-most-one-live structural. **Deliberately not** FK'd to canonical_assets: an estimate matrix is a rate card keyed on asset profiles, a tier above any single asset (a conscious, commented exception to the canonical-asset linkage rule).

estimate_share_access_log

Append-only, mirroring payload_access_log: event ∈ {shared, viewed, revoked, superseded}, dual-party SELECT policies, **no INSERT policy** (service-role _log() only).

Snapshot shape

```
{
  "schema_version": 1,
  "vendor": { "id": "...", "handle": "...", "name": "..." },
  "link_id": "...",
  "granularity": "craft_bucket",
  "variable_fields": ["Asset Type", "Complexity"],
  "unit": "days",
  "generated_at": "2026-05-29T...Z",
  "profiles": [
    { "variable_values": { "Asset Type": "Character", "Complexity": "High" },
      "total_days": 30,
      "breakdown": [ { "label": "Modeling", "days": 12 }, { "label": "Texturing", "days": 18 } ]
  ]
}
```

breakdown is omitted entirely for asset_total; labelled by craft for craft_bucket; labelled by step name for workflow_step.

Atomic share creation

POST `/api/estimate-shares` calls the `create_estimate_share` Postgres RPC, which in one transaction: find-or-creates the `(vendor, link)` series → supersedes any live dispatch (logs `superseded`) → inserts the new dispatch (logs `shared`) → returns the dispatch row. The `uq_esd_one_live` index is the race backstop: a concurrent double-share makes the losing insert fail the unique and the handler retries.

API Reference

All endpoints under `/api/estimate-shares` (JWT-gated).

Vendor (sender)

Method	Path	Description
GET	<code>/targets</code>	Active studio links this vendor can share to
GET	<code>/preview?link_id=&granularity=</code>	Project the effective matrix without persisting, drives the share modal preview; returns <code>{snapshot, unset_cells, profile_count}</code>
POST	<code>``</code>	Create/replace a share: <code>{link_id, granularity, expires_in_days?, label?}</code>
GET	<code>/outbox</code>	Vendor's current (non-superseded) shares, one per channel
POST	<code>/{{dispatch_id}}/revoke</code>	Revoke a share

Studio (recipient)

Method	Path	Description
GET	<code>/inbox</code>	Active shares addressed to this studio (live, not revoked/expired)
POST	<code>/{{dispatch_id}}/view</code>	Record a <code>viewed</code> access-log event

Reads/writes go through the service-role client; the route-layer owner filter (`vendor_id / recipient_studio_id`) is the enforced tenancy boundary. `/{{id}}/view` raises **410** on a revoked or expired share, mirroring payload dispatch.

Code Map

Layer	File
Owner resolution	<code>lib/auth.py</code> <code>resolve_owner()</code>
Effective matrix (base $\oplus$ override)	<code>lib/estimate/effective.py</code>
Projection / granularity boundary	<code>lib/estimate/projector.py</code>
Delivery strategy seam	<code>lib/estimate/delivery.py</code> (<code>route_inbox</code>)
Share endpoints	<code>routes/estimate_share.py</code>
Owner-aware matrix authoring	<code>routes/matrix.py</code> , <code>routes/workflow_steps.py</code>
Vendor override editing	<code>frontend/src/pages/OrgHub.jsx</code> (Estimates tab, studio-link selector)
Vendor share modal + outbox	<code>frontend/src/pages/StudioConnections.jsx</code> , <code>components/ShareEstimatesModal.jsx</code>
Studio inbox viewer	<code>frontend/src/pages/VendorConnections.jsx</code>
Shared snapshot renderer	<code>frontend/src/components/EstimateSnapshotView.jsx</code>
Tests	<code>scripts/test_effective_matrix.py</code> , <code>scripts/test_projector.py</code>

Status & Non-Goals

Done (dev branch): org-scoping (Phase 0), per-link overrides (Phase 1), projection + share API (Phase 2 backend), vendor share UI (M3), studio inbox viewer (M4). Migrations `20260529000001/002/003` applied to dev only, **nothing on prod yet** (prod deploy is a separate gated step). The only open v1 item is a manual vendor-login smoke test of the full chain.

Explicit non-goals for v1 (anti-gold-plating; see the plan for rationale): - No semantic/basis discriminator on the estimate number (no "forecast" vs "quote" type), the number is intentionally semantically open, per-org meaning. - No billing semantics, these are coarse scenario-planning estimate boxes. - No user-facing REFRESH (only the series identity + auto-supersede that enable it). - No scenario-planner / baseline-promotion consumption, the self-describing snapshot keeps it consumable later without migration. - No `estimate_share_field_mappings` vocabulary reconciliation (the studio sees the vendor's own labels) until a studio actually consumes the data. - No profile-subset sharing, v1 shares all profiles in the effective matrix. - No external token delivery strategy, reserved seam only.